

Entrega 1

Alumno: Francisco Pérez Ruiz **Asignatura:** Desarrollo Web en Entorno Servidor (DWES)

Introducción

PHP es un lenguaje que se ejecuta en el servidor. Lo que hace, básicamente, es que cuando el navegador pide una página, el servidor corre el PHP y devuelve HTML. El navegador nunca ve el PHP, solo el HTML resultante. A mí me ayudó pensarlo como un "motor" que mete los datos dentro de la página antes de mandarla.

Esta primera entrega son los ejercicios introductorios del curso. Los temas que se tocan son la sintaxis básica, las variables, los arrays, las funciones, las clases, las cadenas y sus secuencias de escape, las constantes, las fechas y un primer contacto con las superglobales.

Estructura de la entrega

Dentro de la carpeta **Entrega1** están los archivos **Programa1.php** hasta **Programa10_1.php**, este archivo **Entrega1.md** y la carpeta **image/Entrega1** con las capturas que aparecen aquí.

Herramientas que uso

Para no estar recargando todo el rato uso la extensión Live Server, y AutoSave para guardar sin tener que pulsar Ctrl+S. Con Prettier y Mayús+Alt+F se indenta el PHP automáticamente, lo dejo apuntado porque al principio no lo conocía.

Programa 1

Aquí se ve la sintaxis básica de PHP. Lo más importante que apunté:

- `echo` se usa para mostrar texto en el navegador.
- Los comentarios no se ejecutan (con `//` o `/* ... */`).
- Las variables siempre empiezan por `$`. Ejemplo: `$saludo = "¡Hola, Mundo!";`.
- Para los arrays se usan corchetes: `$colores = ["Rojo", "Verde", "Azul"];` y se accede con `$colores[0]`.
- Si una variable está declarada fuera de una función, dentro hay que poner `global` para poder usarla.
- Para concatenar texto se usa el punto `.`, no el `+` como en Java.

Bloque inicial con varios `echo` con colores y un comentario multilinea:

Programa1.php · líneas 1-13

```
1 <?php
2 // Programa 1
3
4 echo "<span style='color: red;'>Programa 1!</span><br>";
5 echo "<span style='color: green;'>Hola 1!</span><br>";
6 echo "<span style='color: blue;'>Esta es la primera línea.</span><br>";
7 echo "<span style='color: orange;'>Esta es la segunda línea.</span><br>";
8 echo "<h1 style='color: red;'>Programa 1 en HTML</h1>";
9
10 /*
11 Este es un comentario
12 de múltiples líneas
13 */
```

Después declaro variables simples y un array, y los muestro:

Programa1.php · líneas 15-29

```
15 // Variables simples
16 $numero = 42;
17 $precio = 19.99;
18 $saludo = "¡Hola, Mundo!";
19 $esVerdadero = true;
20
21 echo "<span style='color: purple;'>$saludo</span><br>";
22 echo "<span style='color: navy;'>Número: $numero</span><br>";
23 echo "<span style='color: navy;'>Float: $precio</span><br>";
24 echo "<span style='color: navy;'>Booleano: $esVerdadero</span><br>";
25
26 // Arrays
27 $colores = ["Rojo", "Verde", "Azul"];
28 echo "<span style='color: teal;'>$colores[0]</span><br>"; // Pintar el elemento 0 del array
29 echo "<span style='color: brown;'>$saludo</span><br>";
```

Aquí está la parte del ámbito de variables (usando `global` dentro de la función) y un pequeño contador:

Programa1.php · líneas 31-51

```
31 //Probando el ambito de las variables
32 $variableGlobal = "Soy una variable global";
33 $x = 10; //Ambito global
34
35 function pruebaAmbito()
36 {
37     global $variableGlobal; // Acceder a la variable global
38     $y = 20; // Ambito local
39     echo "<p>Dentro de la función: $variableGlobal e y = $y</p>";
40 }
41
42 pruebaAmbito();
43
44 //Contador
45 function contador()
46 {
47     $contador = 0;
48     $contador++;
49     echo "<p>Contador: $contador</p>";
50 }
51 contador();
```

También probé a hacer una clase muy simple, `Coche`, con un constructor:

Programa1.php · líneas 53-65

```
53 // Clase Coche
54 class Coche
55 {
56     public $marca;
57
58     public function __construct($marca)
59     {
60         $this->marca = $marca;
61     }
62 }
63
64 $miCoche = new Coche("Toyota");
65 echo "<p>Mi coche es de marca: <strong>" . $miCoche->marca . "</strong></p>";
```

Y un bloque con concatenación y un `foreach` para recorrer un array de frutas:

Programa1.php · líneas 67-79

```
67 // Ejemplo 2: Variables y concatenación
68 $nombre = "Francisco";
69 $edad = 21;
70 echo "<p>Hola, mi nombre es <b>" . $nombre . "</b> y tengo " . $edad . " años.</p>";
71
72 // Ejemplo 3: Arrays y bucles
73 $frutas = ["Manzana", "Pera", "Platano", "Naranja"];
74 echo "<h2>Lista de frutas:</h2>";
75 echo "<ul>";
76 foreach ($frutas as $fruta) {
77     echo "<li>" . $fruta . "</li>";
78 }
79 echo "</ul>";
```

Por último, una función **sumar** y una clase **Persona** que tiene un método **saludar**:

Programa1.php · líneas 81-109

```
81 // Ejemplo 4: Funciones
82 function sumar($a, $b)
83 {
84     return $a + $b;
85 }
86
87 $resultado = sumar(5, 7);
88 echo "<p>La suma es: <strong>" . $resultado . "</strong></p>";
89
90 // Ejemplo 5: Clase Persona
91 class Persona
92 {
93     public $nombre;
94     public $edad;
95
96     public function __construct($nombre, $edad)
97     {
98         $this->nombre = $nombre;
99         $this->edad = $edad;
100     }
101
102     public function saludar()
103     {
104         return "Hola, soy " . $this->nombre . " y tengo " . $this->edad . " años.";
105     }
106 }
107
108 $persona1 = new Persona("Francisco", 21);
109 echo "<p>" . $persona1->saludar() . "</p>";
```

Salida en el navegador:

Programa 1!
Hola 1!
Esta es la primera línea.
Esta es la segunda línea.

Programa 1 en HTML

¡Hola, Mundo!
Número: 42
Float: 19.99
Booleano: 1
Rojo
¡Hola, Mundo!

Dentro de la función: Soy una variable global e y = 20

Contador: 1

Mi coche es de marca: **Toyota**

Hola, mi nombre es **Francisco** y tengo 21 años.

Lista de frutas:

- Manzana
- Pera
- Platano
- Naranja

La suma es: 12

Hola, soy Francisco y tengo 21 años.

Programa 2

Este programa es más corto. Es un HTML con bloques de PHP intercalados. Se declara mi nombre en una variable y se muestra dentro de un `<strong>`. Después se usa `printf` con dos `%s` para encajar el ciclo y el módulo dentro de la frase.

Primer bloque, donde creo la variable y la imprimo:

Programa2.php · líneas 12-20

```
12     <?php
13     // Primer bloque PHP: creamos la variable
14     $nombre = "Francisco Pérez";
15     ?>
16
17     <p>El valor de la variable es:</p>
18
19     <?php echo "<strong>" . $nombre . "</strong>"; ?>
20
```

Segundo bloque, ya con `printf`:

```
21     <?php
22     $ciclo = "DAW";
23     $modulo = "DWES";
24     print "<p>";
25     printf("Estoy estudiando %s en el ciclo de %s", $modulo, $ciclo);
26     print "<p>";
27     ?>
```

Salida en el navegador:

Ejemplo de variable en PHP

El valor de la variable es:

Francisco Pérez

Estoy estudiando DWES en el ciclo de DAW

Programa 3

Variables numéricas y de cadena para representar los datos de una empresa, todo dentro de un único `printf` que mezcla `%s`, `%d` y `%.2f`. Es básicamente el primer ejemplo "real" donde se ve para qué sirve `printf`.

```
11 <?php
12     $nombreEmpresa = "Josmar Inda";
13     $direccion = "Calle los Alamillos";
14     $telefono = "766-899-566";
15     $numEmpleados = 10;
16     $facturacionAnual = 2.5; // 2.5 millones
17     $beneficioNeto = 500000; // 500 mil
18
19     printf("<p>La empresa %s, ubicada en %s, con teléfono %s, cuenta con %d empleados, una facturación anual de %.2f millones y un beneficio neto de %d euros.</p>",
20           $nombreEmpresa, $direccion, $telefono, $numEmpleados, $facturacionAnual, $beneficioNeto);
21 ?>
```

Salida:

Variables de empresa numericas y de cadena

La empresa Josmar Inda, ubicada en Calle los Alamillos, con teléfono 766-899-566, cuenta con 10 empleados, una facturación anual de 2.50 millones y un beneficio neto de 500000 euros.

Programa 4

Aquí se profundiza más con `printf` y `sprintf` aplicados a una aerolínea. La diferencia entre uno y otro es que `printf` imprime directamente y `sprintf` te devuelve la cadena ya formateada para que la uses cuando quieras.

Como referencia rápida me apunto esta tabla con los formatos:

Formato	Para qué sirve	Ejemplo	Resultado
%s	Texto (string)	<code>printf("Hola %s", "Francisco");</code>	Hola Francisco
%d	Entero	<code>printf("Edad: %d", 21);</code>	Edad: 21
%f	Decimal (float)	<code>printf("Precio: %f", 3.5);</code>	Precio: 3.500000
%.2f	Decimal con 2 decimales	<code>printf("Precio: %.2f", 3.5);</code>	Precio: 3.50
%b	Binario	<code>printf("%b", 5);</code>	101
%o	Octal	<code>printf("%o", 9);</code>	11
%x	Hexadecimal (minúscula)	<code>printf("%x", 255);</code>	ff
%X	Hexadecimal (mayúscula)	<code>printf("%X", 255);</code>	FF
%c	Carácter ASCII	<code>printf("%c", 65);</code>	A

Bloque con `printf`:

Programa4.php · líneas 12-49

```
12  <?php
13  $aerolinea = "AirGlobal";
14  $numVuelo = 1205;
15  $precioBase = 245.5;
16  $tasaCombustible = 12.3456;
17  $ocupacion = 87;           // porcentaje de ocupación
18  $codigoInterno = 0b101101;
19  $distancia = 5.6e3;        // 5600 km
20  $velCrucero = 902.456;     // km/h
21
22  // Imprime el nombre de la aerolínea en 10 espacios, alineado a la izquierda
23  printf("Aerolínea: %-10s<br>", $aerolinea);
24
25  // Número de vuelo como entero
26  printf("Vuelo Nº: %d<br>", $numVuelo);
27
28  // Precio base con 2 decimales, en un campo de 8 caracteres
29  printf("Precio base: %8.2f €<br>", $precioBase);
30
31  // Tasa combustible con 4 decimales
32  printf("Tasa combustible: %.4f €<br>", $tasaCombustible);
33
34  // Porcentaje de ocupación (mostrar el símbolo %)
35  printf("Ocupación: %d%%<br>", $ocupacion);
36
37  // Código interno en binario
38  printf("Código interno: %b<br>", $codigoInterno);
39
40  // Distancia en notación científica
41  printf("Distancia: %e km<br>", $distancia);
42
43  // Mezcla de variables en la misma línea (vuelo y velocidad)
44  echo "<br>";
45
46  printf("El vuelo %d de %s vuela a %.3f km/h<br>", $numVuelo, $aerolinea, $velCrucero);
47  echo '<span style="color:red;">Habría que poner tantos formatos como variables haya, lo que va con % son los formatos.</span><br>';
48  sprintf('', $numVuelo);
49  ?>
```

Bloque con **sprintf** (se guarda en **\$mensaje** y luego se imprime con **echo**):

Programa4.php · líneas 51-63

```
51  <?php
52  // Ejemplo con sprintf (genera una cadena formateada sin imprimir)
53  $mensaje = sprintf(
54      "El vuelo %d de %s recorrerá %.1f km con un %d%% de ocupación.",
55      $numVuelo,
56      $aerolinea,
57      $distancia,
58      $ocupacion
59  );
60
61  // Ahora lo mostramos con echo
62  echo "<p><em>Mensaje generado con sprintf:</em> $mensaje</p>";
63  ?>
```

Salida:

Actividad con printf y sprintf

Aerolínea: AirGlobal
Vuelo N°: 1205
Precio base: 245.50 €
Tasa combustible: 12.3456 €
Ocupación: 87%
Código interno: 101101
Distancia: 5.600000e+3 km

El vuelo 1205 de AirGlobal vuela a 902.456 km/h

Habría que poner tantos formatos como variables haya, lo que va con % son los formatos.

Mensaje generado con sprintf: El vuelo 1205 de AirGlobal recorrerá 5600.0 km con un 87% de ocupación.

Programa 5

Trata sobre cómo se comportan las cadenas en PHP. Lo importante:

- Las comillas dobles interpretan secuencias de escape (`\n`, `\t...`) y también las variables que metas dentro.
- Las comillas simples no interpretan nada salvo `\'` y `\\`.
- Se pueden meter códigos Unicode (`\u{2708}`) y hexadecimales (`\x41`).
- `heredoc` es como las dobles pero para texto multilínea, y `nowdoc` es como las simples pero también multilínea.

Primer bloque, con comillas dobles, simples y los códigos:

Programa5.php · líneas 29-38

```
29  <?php
30  // Ejemplos con comillas dobles (interpreta escapes)
31  $dobles = "Hola\nMundo\t(estó está tabulado)\nComilla doble: \" Barra invertida: \\ Variable: \$valor";
32
33  // Ejemplos con comillas simples (no interpreta escapes, salvo \' y \\)
34  $simples = 'Hola\nMundo\t(estó aparece literal)\nComilla simple: \'' Barra invertida: \\ Variable: $valor';
35
36  // Unicode y Hex
37  $unicode = "Avión: \u{2708} (símbolo avión)";
38  $hex = "Códigos hexadecimales: \x41\x42\x43 = ABC";
```

Segundo bloque, con heredoc y nowdoc:

Programa5.php · líneas 40-55

```
40  // Heredoc (interpreta escapes)
41  $heredoc = <<<TEXT
42  Cadena con heredoc:
43  - Nueva línea \n (se interpreta)
44  - Tabulación \t (se interpreta)
45  - Variable \$unicode: $unicode
46  TEXT;
47
48  // Nowdoc (no interpreta escapes)
49  $nowdoc = <<<'TEXT'
50  Cadena con nowdoc:
51  - Nueva línea \n (literal)
52  - Tabulación \t (literal)
53  - Variable $unicode (no sustituida)
54  TEXT;
55  ?>
```

Salida:

Estudio de cadenas y secuencias de escape en PHP

Con comillas dobles

```
Hola
Mundo    (esto está tabulado)
Comilla doble: " Barra invertida: \ Variable: $valor
```

Con comillas simples

```
Hola\nMundo\t(estos aparecen literales)\nComilla simple: ' Barra invertida: \ Variable: $valor
```

Unicode y Hex

```
Avión: ✈ (símbolo avión)
Códigos hexadecimales: ABC = ABC
```

Heredoc

```
Cadena con heredoc:
- Nueva línea
  (se interpreta)
- Tabulación    (se interpreta)
- Variable $unicode: Avión: ✈ (símbolo avión)
```

Nowdoc

```
Cadena con nowdoc:
- Nueva línea \n (literal)
- Tabulación \t (literal)
- Variable $unicode (no sustituida)
```

Programa 6

Aquí trabajamos con tipos. Las funciones que me hicieron falta fueron **gettype** para ver el tipo, **settype** para forzarlo, **is_string** / **is_array** / etc. para comprobar, **isset** para saber si la variable existe y **unset** para borrarla.

Una cosa importante: las constantes **no llevan \$** delante.

Bloque de declaración y **gettype**:

Programa6.php · líneas 20-32

```
20 // 1. Declaración de variables
21 $cadena = "Hola mundo";
22 $entero = 25;
23 $decimal = 12.34;
24 $lista = array("avión", "helicóptero", "dron");
25 $nulo = null;
26 $cadenaPrueba = "Hola mundo2";
27 settype($cadenaPrueba, "float");
28
29 // 2. Mostrar tipo con gettype
30 echo "Tipo de cadena: " . gettype($cadena) . "<br>";
31 echo "Tipo de entero: " . gettype($entero) . "<br>";
32 echo "Tipo de cadenaPrueba: " . gettype($cadenaPrueba) . " $cadenaPrueba <br>";
```

Comprobaciones con `is_*` y conversión con `settype`:

Programa6.php · líneas 33-44

```
33 // 3. Comprobaciones con is_...
34 if (is_string($cadena)) {
35     echo "La variable es una cadena<br>";
36 }
37 if (is_array($lista)) {
38     echo "La variable es un array<br>";
39 }
40
41 // 4. Conversión con settype
42 echo "Antes de convertir: " . gettype($decimal) . "<br>";
43 settype($decimal, "integer");
44 echo "Después de convertir: " . gettype($decimal) . "<br>";
```

`isset` y `unset` para comprobar y borrar la variable:

Programa6.php · líneas 46-56

```
46 // 5. isset
47 if (isset($lista)) {
48     echo "La variable está definida y no es null<br>";
49 }
50
51 // 6. unset
52 unset($entero);
53 echo "Después de unset: " . (isset($entero) ? "existe" : "no existe") . "<br>";
54
55 // 7 mostramos variable No definida
56 echo "<br>Mostramos variable no definida antes de desactivar los Warnings:$entero"; // Notice: Undefined variable: entero
```

Si al final del programa intento imprimir una variable que ya no existe, sale un Warning. Eso se puede ocultar con `error_reporting(0)` o desde el `php.ini`:

```

60 // Oculta todos los warnings (aunque se sigan generando internamente)
61 error_reporting(0); // 7 mostramos variable No definida
62 echo "<br>Mostramos variable no definida despues de desactivar los Warnings:$entero"; // Notice: Undefined variable: entero
63 ini_set('display_errors', 0);
64
65 echo $variableInexistente; // No muestra nada en pantalla
66

```

Salida:

Tipo de cadena: string
 Tipo de entero: integer
 Tipo de cadenaPrueba: double0
 La variable es una cadena
 La variable es un array
 Antes de convertir: double
 Después de convertir: integer
 La variable está definida y no es null
 Después de unset: no existe

Warning: Undefined variable \$entero in C:\xampp\htdocs\DWES_FranciscoPerez\UD2_FuncionesFormularios\Entrega1\Programa6.php on line 56

Mostramos variable no definida antes de desactivar los Warnings:
 Mostramos variable no definida despues de desactivar los Warnings:

Programa 7

Constantes con **define** y con **const**. Parecen lo mismo pero no lo son:

- **define()** se ejecuta en tiempo de ejecución, por eso se puede meter dentro de un **if**.
- **const** se evalúa en tiempo de compilación, no admite expresiones complicadas ni vivir dentro de un **if**.

Bloque con **define**:

Programa7.php · líneas 1-11

```
1 <?php
2 //Definir constantes con define
3 define("PI", 3.14);
4 print "<p>El valor de pi es " . PI . "</p>\n";
5
6 define("AUTOR", "Echiro Oda");
7 print "<p>Autor: " . AUTOR . "</p>\n";
8
9 define("LIBRO", ["One Piece", "Echiro Oda", 1995]);
10 print "<p>" . LIBRO[1] . " escribió " . LIBRO[0] . " en " . LIBRO[2] . ".</p>\n";
11
```

Bloque con **const**:

Programa7.php · líneas 13-21

```
13 //Definir constantes con const
14 const PIconst = 3.14;
15 print "<p>El valor de pi es " . PI . "</p>\n";
16
17 const AUTORconst = "Echiro Oda";
18 print "<p>Autor: " . AUTOR . "</p>\n";
19
20 const LIBROconst = ["One Piece", "Echiro Oda", 1995];
21 print "<p>" . LIBRO[1] . " escribió " . LIBRO[0] . " en " . LIBRO[2] . ".</p>\n";
```

Reasignaciones y **count(get_defined_constants())** para ver cuántas hay definidas:

Programa7.php · líneas 23-48

```
23 define("PI", 3.14);
24 print "<p>El valor de pi es PI</p>\n";           // El valor NO se sustituye
25 print "<p>El valor de pi es {PI}</p>\n";         // El valor NO se sustituye
26 print "<p>El valor de pi es " . PI . "</p>\n";   // El valor SÍ se sustituye
27
28 define("PI", 3.14);
29 define("pi", 3.141592);
30 print "<p>El valor de pi es " . PI . "</p>";
31 print "<p>El valor de pi es " . pi . "</p>";
32
33 define("PI", 3.14);
34 define("PI", 3.141592);
35 print "<p>El valor de pi es " . PI . "</p>";
36
37
38 $decimales = 6;
39 if ($decimales == 6) {
40     define("PI", 3.141592);
41 } else {
42     define("PI", 3.14);
43 }
44 print "<p>El valor de pi es " . PI . "</p>\n";
45
46 echo "<pre>\n";
47 echo "Número de constantes definidas: " . count(get_defined_constants()) . "\n";
48 echo "</pre>\n";
```

Y al final hay un bloque que está comentado porque, si se ejecuta, da Fatal Error: usar `const` dentro de un `if` o asignarle una variable. Lo dejo para acordarme:

Programa7.php · líneas 50-64

```
50 /*
51 $decimales = 6;
52
53 if ($decimales == 6) {
54     const PI = 3.141592;
55 } else {
56     const PI = 3.14;
57 }
58
59 print "<p>El valor de pi es " . PI . "</p>\n";
60
61 $a = 5;
62 const CINCO = $a;
63 print "<p>CINCO es " . CINCO . "</p>\n";
64 */
```


Salida:

El valor de pi es 3.14

Autor: Echiro Oda

Echiro Oda escribió One Piece en 1995.

El valor de pi es 3.14

Autor: Echiro Oda

Echiro Oda escribió One Piece en 1995.

Warning: Constant PI already defined in C:\xampp\htdocs\DWES_FranciscoPerez\UD2_FuncionesFormularios\Entregal\Programa7.php on line 23

El valor de pi es PI

El valor de pi es {PI}

El valor de pi es 3.14

Warning: Constant PI already defined in C:\xampp\htdocs\DWES_FranciscoPerez\UD2_FuncionesFormularios\Entregal\Programa7.php on line 28

El valor de pi es 3.14

El valor de pi es 3.141592

Warning: Constant PI already defined in C:\xampp\htdocs\DWES_FranciscoPerez\UD2_FuncionesFormularios\Entregal\Programa7.php on line 33

Warning: Constant PI already defined in C:\xampp\htdocs\DWES_FranciscoPerez\UD2_FuncionesFormularios\Entregal\Programa7.php on line 34

El valor de pi es 3.14

Warning: Constant PI already defined in C:\xampp\htdocs\DWES_FranciscoPerez\UD2_FuncionesFormularios\Entregal\Programa7.php on line 40

El valor de pi es 3.14

Número de constantes definidas: 1885

Programa 8

Es una mezcla de PHP y HTML. La parte de PHP define unas constantes (deporte, equipo, estadio, aforo) y unas variables de ejemplo. Después, ya en HTML, se usan tablas con `<?= ... ?>` para volcar las constantes y comprobar su tipo con `gettype`, `is_string` y `is_int`.

Parte PHP:

```
1 <?php
2 // --- Constantes propias ---
3 const DEPORTE = "Calistenio";
4 const EQUIPO = "LZKREW";
5 const ESTADIO = "Gran Arena";
6 const AFORO = 50000;
7
8 // --- Constantes predefinidas de PHP ---
9 $constantes_php = [
10     "PHP_VERSION" => PHP_VERSION,
11     "PHP_OS" => PHP_OS,
12     "__FILE__" => __FILE__,
13 ];
14
15 // --- Constantes propias en array para analizarlas ---
16 $constantes_propias = [
17     "deporte" => DEPORTE,
18     "equipo" => EQUIPO,
19     "estadio" => ESTADIO,
20     "aforo" => AFORO,
21 ];
22
23 // --- Variables de ejemplo ---
24 $jugadores = ["Pedro", "Francisco", "Luis"]; // array
25 $goles = 3; // entero
26 $capitan = "Carlos"; // string
27 $esLocal = true; // booleano
28 ?>
```

Parte HTML con las tablas:

```

30 <!DOCTYPE html>
31 <html lang="es">
32 <head>
33   <meta charset="UTF-8">
34   <title><?= EQUIPO ?> - <?= DEPORTE ?></title>
35   <style>
36     table { border-collapse: collapse; margin: 1em 0; }
37     th, td { border: 1px solid #999; padding: .4em .8em; }
38     th { background: #eee; }
39   </style>
40 </head>
41 <body>
42   <h1><?= EQUIPO ?> (<?= DEPORTE ?>)</h1>
43   <p>Estadio: <strong><?= ESTADIO ?></strong></p>
44   <p>Aforo máximo: <strong><?= AFORO ?></strong> espectadores</p>
45
46   <h2>Constantes de PHP</h2>
47   <table>
48     <tr>
49       <th>Nombre</th><th>Valor</th><th>gettype</th><th>is_string</th><th>is_int</th>
50     </tr>
51     <?php foreach ($constantes_php as $nombre => $valor): ?>
52       <tr>
53         <td><?= $nombre ?></td>
54         <td><?= $valor ?></td>
55         <td><?= getType($valor) ?></td>
56         <td><?= is_string($valor) ? "sí" : "no" ?></td>
57         <td><?= is_int($valor) ? "sí" : "no" ?></td>
58       </tr>
59     <?php endforeach; ?>
60   </table>
61
62   <h2>Constantes propias</h2>
63   <table>
64     <tr>
65       <th>Nombre</th><th>Valor</th><th>gettype</th><th>is_string</th><th>is_int</th>
66     </tr>
67     <?php foreach ($constantes_propias as $nombre => $valor): ?>
68       <tr>
69         <td><?= $nombre ?></td>
70         <td><?= $valor ?></td>
71         <td><?= getType($valor) ?></td>
72         <td><?= is_string($valor) ? "sí" : "no" ?></td>
73         <td><?= is_int($valor) ? "sí" : "no" ?></td>
74       </tr>
75     <?php endforeach; ?>
76   </table>

```

LZKREW (Calistenio)

Estadio: Gran Arena

Aforo máximo: 50000 espectadores

Constantes de PHP

Nombre	Valor	gettype	is_string	is_int
PHP_VERSION	8.2.12	string	sí	no
PHP_OS	WINNT	string	sí	no
__FILE__	C:\xampp\htdocs\DWES_FranciscoPerez\UD2_FuncionesFormularios\Entrega1\Programa8.php	string	sí	no

Constantes propias

Nombre	Valor	gettype	is_string	is_int
deporte	Calistenio	string	sí	no
equipo	LZKREW	string	sí	no
estadio	Gran Arena	string	sí	no
aforo	50000	integer	no	sí

Variables de ejemplo

Nombre	Valor	gettype	is_array	is_integer	is_string	is_bool
\$jugadores	Pedro, Francisco, Luis	array	sí	no	no	no
\$goles	3	integer	no	sí	no	no
\$capitan	Carlos	string	no	no	sí	no
\$esLocal	true	boolean	no	no	no	sí

Programa 9

Cómo mostrar la fecha y la hora con `date()`. Cambiando las letras del formato se sacan combinaciones distintas. Por ejemplo, `d/m/Y H:i:s` me da la fecha y hora completas. Algunas que aprendí:

- `l` para el nombre del día de la semana.
- `F` para el mes en texto.
- `W` para la semana del año.
- `z` para el día del año.
- `A` para AM/PM.

Primer bloque (fecha y hora normales, día y mes):

Programa9.php · líneas 1-18

```
1 <?php
2 // Fecha y hora actual
3 echo "<p>Fecha y hora actual: " . date("d/m/Y H:i:s") . "</p>";
4
5 // Solo la fecha
6 echo "<p>Fecha de hoy: " . date("d-m-Y") . "</p>";
7
8 // Solo la hora
9 echo "<p>Hora actual: " . date("H:i:s") . "</p>";
10
11 // Día de la semana
12 echo "<p>Día de la semana: " . date("l") . "</p>";
13
14 // Día, mes y año por separado
15 echo "<p>Día: " . date("d") . "</p>";
16 echo "<p>Mes: " . date("F") . "</p>"; // Nombre completo del mes
17 echo "<p>Mes abreviado: " . date("M") . "</p>";
18 echo "<p>Año: " . date("Y") . "</p>";
```

Segundo bloque (semana del año, día del año, AM/PM y timestamp):

Programa9.php · líneas 20-31

```
20 // Semana del año
21 echo "<p>Semana del año: " . date("W") . "</p>";
22
23 // Día del año
24 echo "<p>Día del año: " . date("z") . "</p>";
25
26 // AM/PM
27 echo "<p>Hora con AM/PM: " . date("h:i:s A") . "</p>";
28
29 // Timestamp actual
30 echo "<p>Timestamp actual: " . time() . "</p>";
31 ?>
```

Salida:

Fecha y hora actual: 09/06/2026 16:36:49

Fecha de hoy: 09-06-2026

Hora actual: 16:36:49

Día de la semana: Tuesday

Día: 09

Mes: June

Mes abreviado: Jun

Año: 2026

Semana del año: 24

Día del año: 159

Hora con AM/PM: 04:36:49 PM

Timestamp actual: 1781015809

Programa 10

Primer contacto con las superglobales y las constantes predefinidas.

Las superglobales que vimos aquí son `$_SERVER`, que tiene info del servidor y la petición, y `$_GET`, que recoge los parámetros pasados por la URL. Con el operador `??` se puede dar un valor por defecto si la clave no existe.

Las constantes predefinidas más útiles son `PHP_VERSION`, `PHP_OS`, `PHP_INT_MAX` y `DIRECTORY_SEPARATOR`.

Acceso a `$_SERVER` y `$_GET`:

code_prog10_1

Acceso a las constantes predefinidas:

```
17 <h2>Constantes Predefinidas</h2>
18 <?php
19 echo "<p>Versión de PHP: " . PHP_VERSION . "</p>";
20 echo "<p>Sistema operativo del servidor: " . PHP_OS . "</p>";
21 echo "<p>Directorio de instalación de PHP: " . PHP_BINDIR . "</p>";
22 echo "<p>Tamaño máximo de enteros (PHP_INT_MAX): " . PHP_INT_MAX . "</p>";
23 echo "<p>Separador de directorios (DIRECTORY_SEPARATOR): " . DIRECTORY_SEPARATOR . "</p>";
24 ?>
```

Salida:

Variables Predefinidas (Superglobales)

Servidor: localhost

Archivo actual: /DWES_FranciscoPerez/UD2_FuncionesFormularios/Entrega1/Programa10.php

Dirección IP del cliente: ::1

Parámetro GET 'ejemplo': No enviado

Constantes Predefinidas

Versión de PHP: 8.2.12

Sistema operativo del servidor: WINNT

Directorio de instalación de PHP: C:\php

Tamaño máximo de enteros (PHP_INT_MAX): 9223372036854775807

Separador de directorios (DIRECTORY_SEPARATOR): \

Programa 10_1

Versión ampliada del anterior. Aquí se imprimen prácticamente todas las superglobales una detrás de otra: `$_SERVER`, `$_GET`, `$_POST`, `$_REQUEST`, `$_FILES`, `$_ENV`, `$_GLOBALS`, `$_COOKIE` y `$_SESSION`. Sirve como "panel de control" para entender qué contiene cada una. También tiene dos formularios, uno GET y otro POST, para ver cómo cambian las superglobales según el método.

Primera parte (\$_SERVER, \$_GET y \$_POST):

Programa10_1.php · líneas 10-25

```
10  <h2>1. $_SERVER</h2>
11  <pre><?php print_r( $_SERVER ); ?></pre> <!-- HUECO 1 -->
12
13  <h2>2. $_GET</h2>
14  <form method="get">
15      <label>Nombre (GET): <input type="text" name="nombre"></label>
16      <input type="submit" value="Enviar GET">
17  </form>
18  <pre><?php print_r( $_GET ); ?></pre> <!-- HUECO 2 -->
19
20  <h2>3. $_POST</h2>
21  <form method="post">
22      <label>Edad (POST): <input type="number" name="edad"></label>
23      <input type="submit" value="Enviar POST">
24  </form>
25  <pre><?php print_r( $_POST ); ?></pre> <!-- HUECO 3 -->
```

Segunda parte (resto de superglobales):

Programa10_1.php · líneas 27-49

```
27  <h2>4. $_REQUEST</h2>
28  <pre><?php print_r( $_REQUEST ); ?></pre> <!-- HUECO 4 -->
29
30  <h2>5. $_FILES</h2>
31  <form method="post" enctype="multipart/form-data">
32      <label>Subir archivo: <input type="file" name="archivo"></label>
33      <input type="submit" value="Enviar Archivo">
34  </form>
35  <pre><?php print_r( $_FILES ); ?></pre> <!-- HUECO 5 -->
36
37  <h2>6. $_ENV</h2>
38  <pre><?php print_r( $_ENV ); ?></pre> <!-- HUECO 6 -->
39
40  <h2>7. $_GLOBALS</h2>
41  <pre><?php print_r( $_GLOBALS ); ?></pre> <!-- HUECO 7 -->
42
43  <h2>8. $_COOKIE</h2>
44  <pre><?php print_r( $_COOKIE ); ?></pre> <!-- HUECO 8 -->
45
46  <h2>9. $_SESSION</h2>
47  <pre><?php print_r( $_SESSION ); ?></pre> <!-- HUECO 9 -->
48
49  </body>
```

Salida:

Índice	1
1. Introducción	2
2. Instalación de PHP	3
3. Configuración de PHP	4
4. Variables	5
5. Tipos de datos	6
6. Operadores	7
7. Estructuras de control	8
8. Funciones	9
9. Arrays	10
10. Manejo de archivos	11
11. Manejo de bases de datos	12
12. Manejo de sesiones	13
13. Manejo de cookies	14
14. Manejo de formularios	15
15. Manejo de errores	16
16. Manejo de excepciones	17
17. Manejo de recursos	18
18. Manejo de sockets	19
19. Manejo de streams	20
20. Manejo de procesos	21
21. Manejo de pipes	22
22. Manejo de buffers	23
23. Manejo de hilos	24
24. Manejo de mutexes	25
25. Manejo de semáforos	26
26. Manejo de variables globales	27
27. Manejo de variables locales	28
28. Manejo de variables de entorno	29
29. Manejo de variables de configuración	30
30. Manejo de variables de sistema	31
31. Manejo de variables de usuario	32
32. Manejo de variables de aplicación	33
33. Manejo de variables de entorno de trabajo	34
34. Manejo de variables de entorno de ejecución	35
35. Manejo de variables de entorno de red	36
36. Manejo de variables de entorno de red de área local	37
37. Manejo de variables de entorno de red de área amplia	38
38. Manejo de variables de entorno de red de área metropolitana	39
39. Manejo de variables de entorno de red de área global	40
40. Manejo de variables de entorno de red de área mundial	41
41. Manejo de variables de entorno de red de área universal	42
42. Manejo de variables de entorno de red de área cósmica	43
43. Manejo de variables de entorno de red de área galáctica	44
44. Manejo de variables de entorno de red de área intergaláctica	45
45. Manejo de variables de entorno de red de área extragaláctica	46
46. Manejo de variables de entorno de red de área extragaláctica	47
47. Manejo de variables de entorno de red de área extragaláctica	48
48. Manejo de variables de entorno de red de área extragaláctica	49
49. Manejo de variables de entorno de red de área extragaláctica	50
50. Manejo de variables de entorno de red de área extragaláctica	51
51. Manejo de variables de entorno de red de área extragaláctica	52
52. Manejo de variables de entorno de red de área extragaláctica	53
53. Manejo de variables de entorno de red de área extragaláctica	54
54. Manejo de variables de entorno de red de área extragaláctica	55
55. Manejo de variables de entorno de red de área extragaláctica	56
56. Manejo de variables de entorno de red de área extragaláctica	57
57. Manejo de variables de entorno de red de área extragaláctica	58
58. Manejo de variables de entorno de red de área extragaláctica	59
59. Manejo de variables de entorno de red de área extragaláctica	60
60. Manejo de variables de entorno de red de área extragaláctica	61
61. Manejo de variables de entorno de red de área extragaláctica	62
62. Manejo de variables de entorno de red de área extragaláctica	63
63. Manejo de variables de entorno de red de área extragaláctica	64
64. Manejo de variables de entorno de red de área extragaláctica	65
65. Manejo de variables de entorno de red de área extragaláctica	66
66. Manejo de variables de entorno de red de área extragaláctica	67
67. Manejo de variables de entorno de red de área extragaláctica	68
68. Manejo de variables de entorno de red de área extragaláctica	69
69. Manejo de variables de entorno de red de área extragaláctica	70
70. Manejo de variables de entorno de red de área extragaláctica	71
71. Manejo de variables de entorno de red de área extragaláctica	72
72. Manejo de variables de entorno de red de área extragaláctica	73
73. Manejo de variables de entorno de red de área extragaláctica	74
74. Manejo de variables de entorno de red de área extragaláctica	75
75. Manejo de variables de entorno de red de área extragaláctica	76
76. Manejo de variables de entorno de red de área extragaláctica	77
77. Manejo de variables de entorno de red de área extragaláctica	78
78. Manejo de variables de entorno de red de área extragaláctica	79
79. Manejo de variables de entorno de red de área extragaláctica	80
80. Manejo de variables de entorno de red de área extragaláctica	81
81. Manejo de variables de entorno de red de área extragaláctica	82
82. Manejo de variables de entorno de red de área extragaláctica	83
83. Manejo de variables de entorno de red de área extragaláctica	84
84. Manejo de variables de entorno de red de área extragaláctica	85
85. Manejo de variables de entorno de red de área extragaláctica	86
86. Manejo de variables de entorno de red de área extragaláctica	87
87. Manejo de variables de entorno de red de área extragaláctica	88
88. Manejo de variables de entorno de red de área extragaláctica	89
89. Manejo de variables de entorno de red de área extragaláctica	90
90. Manejo de variables de entorno de red de área extragaláctica	91
91. Manejo de variables de entorno de red de área extragaláctica	92
92. Manejo de variables de entorno de red de área extragaláctica	93
93. Manejo de variables de entorno de red de área extragaláctica	94
94. Manejo de variables de entorno de red de área extragaláctica	95
95. Manejo de variables de entorno de red de área extragaláctica	96
96. Manejo de variables de entorno de red de área extragaláctica	97
97. Manejo de variables de entorno de red de área extragaláctica	98
98. Manejo de variables de entorno de red de área extragaláctica	99
99. Manejo de variables de entorno de red de área extragaláctica	100
100. Manejo de variables de entorno de red de área extragaláctica	101

Conclusiones de la entrega

Lo más importante que me llevo de esta primera entrega:

- Con `echo` y `printf` cubro casi cualquier salida de texto.
- Las funciones de tipos (`gettype`, `settype`, `isset`, `unset`) son básicas para no quedarme bloqueado cuando salgan Warnings raros.
- `define` y `const` no son intercambiables aunque lo parezcan.
- Las superglobales son el puente entre el navegador y mi código PHP.

Como nota personal: el Programa 6 fue el que más me hizo entender por qué hay errores tan típicos cuando se trabaja con variables sin comprobar. Vale la pena invertir tiempo aquí antes de seguir.